

Sales Revenue Forecasting on Microsoft Fabric

Project manual for **Revenue_Forecast_Notebook.ipynb** — the machine-learning layer on top of the Bronze → Silver → Gold lakehouse pipeline

Version 2 · 10 September 2026

Contents

- 1. Overview and architecture
- 2. Data: sources, revenue definition, profile, known quality issues
- 3. Notebook walkthrough (cell by cell) and parameters
- 4. Methodology: models, features, scaling, intervals, metrics, back-test
- 5. Results on the current data
- 6. Output tables (schema reference)
- 7. MLflow tracking and registered models
- 8. How to run: Fabric portal, pipeline, VS Code (Fabric runtime or local kernel)
- 9. Power BI
- 10. Operations, monitoring and troubleshooting
- 11. Roadmap / ideas
- Appendix A – parameter reference · Appendix B – local requirements · Appendix C – full notebook source

1. Overview and architecture

The Fabric workspace already implements a medallion architecture: raw sales CSV files land in the **Bronze** lakehouse, **Silver_Notebook** cleans them into **Silver.dbo.sales_silver** (upsert), and **Gold_Notebook** builds a star schema in the **Gold** lakehouse (dimcustomer_gold, dimproduct_gold, dimdate_gold, factsales_gold). A pipeline runs Silver → Gold. This project adds a third notebook that trains and benchmarks forecasting models on the Gold fact table and writes a weekly and monthly revenue forecast back to Gold, so Power BI can show history and forecast side by side.

```
Bronze (sales csv) --Silver_Notebook--> Silver.sales_silver --Gold_Notebook--> Gold.factsales_gold + dims
                                     |
                                     Revenue_Forecast_Notebook (this project)
                                     v
Gold.revenue_forecast_gold           daily forecast of the latest run
Gold.revenue_weekly_gold             actual + forecast revenue per ISO week (history + horizon)
Gold.revenue_monthly_gold            actual + forecast revenue per calendar month (history + horizon)
Gold.forecast_backtest_gold          benchmark of every candidate model (per model / fold / grain)
Gold.revenue_forecast_history_gold    monthly forecast of every run (append-only) -> accuracy over time
MLflow experiment sales_revenue_forecast + registered models sales_revenue_forecaster_<component>
```

Deliverables

- **Revenue_Forecast_Notebook.ipynb** – the notebook (imports into Fabric with the Gold lakehouse pre-attached; also runs in VS Code).
- **README_VSCode.md** and **requirements-local.txt** – running it from VS Code on Fabric Spark or on a local Python kernel.
- This manual.

Design principles

- Everything reads the *live* lakehouse tables – no exports, no copies. Each run sees the latest committed version of factsales_gold.
- Model choice is evidence-based: nine candidates are back-tested on every run and the best one (the *champion*) is retrained and used. As data grows the champion can change.
- Cheap on trial capacity: the aggregation to one row per day happens in Spark; the modelling works on ~900 daily / ~130 weekly points in pandas and takes seconds.
- Same code in Fabric and on a laptop: the notebook detects whether a Spark session exists and switches between Spark and delta-rs/OneLake access.

2. Data

2.1 Source tables (Gold lakehouse, schema dbo)

Table	Columns	Notes
factsales_gold	CustomerID (long), ItemID (long), OrderDate (date), Quantity (int), UnitPrice (float), Tax (float)	Fact table, merge key OrderDate + CustomerID + ItemID. Input of the forecast.
dimdate_gold	OrderDate, Day, Month, Year, mmmmyyyy, yyyyymm	Only dates that have sales. The forecast tables carry the same yyyyymm / mmmmyyyy keys.
dimcustomer_gold	CustomerName, Email, First, Last, CustomerID	Not used by the forecast.
dimproduct_gold	ItemName, ItemID, ItemInfo	Not used yet – needed for per-category forecasts (see roadmap).

2.2 Revenue definition

Revenue = Quantity × UnitPrice + Tax (parameter **INCLUDE_TAX = True**; set it to False for net revenue). Revenue is summed per OrderDate to a daily series; days without orders are filled with 0 so that the calendar is continuous.

2.3 Profile of the current data

Year	Days with sales	Order lines	Revenue incl. tax
2019	184	1,201	4,172,170
2020	365	2,733	6,882,259
2021	365	28,784	11,547,835

Facts that shaped the modelling decisions:

- History runs from 1 July 2019 to 31 December 2021 (915 days, 130 complete weeks, 30 months). Only 2.5 years, so yearly seasonality is weakly identified.
- Quantity is always 1; every line is one unit. Bikes (Mountain-/Road-/Touring-) are about 95 % of revenue.
- A structural break in June–July 2021: bike volume roughly triples and accessories/clothing appear (22.5k low-value lines). Monthly revenue moves from ~0.5 M to 1.3–1.8 M.
- There is no day-of-week pattern (synthetic data), so weekly/daily seasonality is switched off everywhere and models are scored on weekly and monthly totals.
- November is the strongest month in each year; August–October 2020 was a trough.

2.4 Known data-quality issues upstream

- **Date formats differ per file:** 2019/2020 use yyyy-MM-dd, 2021 uses M/d/yyyy. The Silver notebook reads all files with one DateType schema; if a runtime ever parses the 2021 dates as null, the whole year disappears from sales_silver. The forecast notebook prints a rows-per-year check so this is caught immediately.
- One malformed line (SalesOrderNumber S089383) has an unparseable date and is dropped.
- factsales_gold has no order-line key, so two lines of the same customer / item / day collapse into one row (5 rows, about 81 currency units plus tax in this data – negligible, but worth a note in the Gold merge).

3. Notebook walkthrough

The notebook has 8 numbered sections. Markdown cells explain each step inline; this is the map.

#	Cell / section	What it does
0	Dependency bootstrap	Installs Prophet if it is missing (Fabric runtime lacks it). Must stay the first cell: in Fabric %pip restarts the interpreter.
0	Parameters	All knobs in one place (table names, horizon, tax, folds, interval width, MLflow options). Mark it as the parameter cell to override from a pipeline.
1	Connect	Detects RUN_MODE (fabric-spark / local). Defines read_fact_daily() and write_table() that use Spark in Fabric and delta-rs + azure-identity on OneLake locally.
1	MLflow setup	Fabric: workspace experiment. Local: Fabric experiment through synapseml-mlflow (sds:// tracking URI) or ./mlruns fallback.
2	History	Builds the daily series, computes the forecast window (rest of the current month + HORIZON_MONTHS full months), prints rows per year, weekly (ISO Mon–Sun) and monthly actuals.
3	Candidate models	Defines the six components, the nine candidates (components and blends), the interval logic and the metric function.
4	Benchmark	Rolling-origin back-test: BACKTEST_FOLDS folds, each trained to a month-end and scored on the next 3 months at weekly and monthly grain. Leaderboard, champion, one MLflow run per candidate.
5	Forecast	Retrains the champion on all history, forecasts daily, builds the weekly/monthly actual+forecast tables, logs the champion run, models and CSV artefacts.
6	Write	Overwrites the four result tables and appends this run's monthly forecast to the history table.
7	Charts	Weekly line with interval band, monthly bars with error bars.
8	Next steps	Pipeline, Power BI, VS Code, extensions.

Parameters are listed in Appendix A.

4. Methodology

4.1 Candidate models

Candidate	Type	Trained on	Scaling / transform	Prediction interval
naive_last_week	baseline	weekly totals	–	empirical h-week spread
moving_avg_4w	baseline (mean of last 4 weeks)	weekly totals	–	empirical h-week spread
holt_damped	Holt damped-trend exponential smoothing (statsmodels)	weekly totals	scale-invariant	500 simulated paths, quantiles

Candidate	Type	Trained on	Scaling / transform	Prediction interval
prophet	piecewise-linear trend with changepoints + yearly seasonality (Meta Prophet)	daily revenue	scale-invariant; log target was tested and hurt	Prophet's own interval
ridge_lags	Ridge regression on lagged weekly growth rates	weekly log-growth	StandardScaler in the sklearn pipeline (required for a penalised linear model) + log1p target	empirical h-week spread
lightgbm_lags	gradient-boosted trees on the same features	weekly log-growth	StandardScaler for consistency (trees are insensitive) + log1p target	empirical h-week spread
prophet_holt_blend	average of prophet and holt_damped	–	–	average of component intervals
prophet_ridge_blend	average of prophet and ridge_lags	–	–	average of component intervals
prophet_holt_ridge_blend	average of prophet, holt_damped and ridge_lags	–	–	average of component intervals

4.2 Features of the ML models

Both feature-based models predict next week's **log-growth** $g_t = \log(1+y_t) - \log(1+y_{t-1})$ of weekly revenue and are rolled forward recursively over the horizon. Features: **growth_lag1 ... growth_lag4, growth_lag8** (the previous growth rates). A rolling mean of the last four growth rates was dropped because it is an exact linear combination of the four lags (perfect collinearity, $VIF = \infty$) and forecasts were identical without it; the remaining features are only mildly correlated (adjacent lags about -0.4 , $VIF \leq 1.7$), so ridge's penalty is a mild stabiliser for 120 training rows rather than a collinearity fix. Calendar features (week-of-year sine/cosine, month) and level-versus-trend features were implemented and benchmarked but made the models worse on this data – with a single prior year they learn 2020's idiosyncrasies (e.g. the autumn 2020 trough) and project them onto 2021 – so they are left out of the default feature list (the code still computes them for experiments). Modelling growth rather than the level makes the target stationary, removes the scale of revenue from the problem and lets the models extrapolate the 2021 growth, which tree models on raw lags cannot.

4.3 Prophet configuration

growth = linear, changepoint_prior_scale = 0.5 (flexible – the series has a large level shift), changepoint_range = 0.9, yearly seasonality on, weekly and daily seasonality off. These were chosen from a grid (0.05 / 0.2 / 0.5 / 1.0, with and without yearly seasonality, additive vs multiplicative, raw vs log) on rolling back-tests before the notebook was written.

4.4 Prediction intervals

Interval width is a parameter (INTERVAL_WIDTH = 0.90). Prophet and Holt provide their own intervals. For the baselines and the ML models the notebook measures, on the training window, the standard deviation of the h-week-ahead change in log revenue for $h = 1 \dots 52$ and applies it multiplicatively ($\hat{y} \times \exp(\pm z \cdot s_h)$). This is deliberately not a random-walk ($\sqrt{h}$) rule: weekly totals are noisy but the noise does not accumulate, and $\sqrt{h}$ intervals were three times too wide. Summed daily bounds in the weekly/monthly tables are conservative (they ignore averaging within the period).

4.5 Metrics

All metrics are computed on period totals (weekly bins of 7 days and calendar months) in the fold's horizon, then averaged over folds:

Metric	Definition	Reading
MAE	mean forecast – actual	in revenue units
RMSE	$\sqrt{\text{mean}(\text{forecast} - \text{actual})^2}$	penalises large misses
MAPE	$\text{mean}(\text{forecast} - \text{actual} / \text{actual}) \times 100$	per-period percentage error; unstable for small actuals
sMAPE	$\text{mean } 2 \text{forecast} - \text{actual} / (\text{actual} + \text{forecast}) \times 100$	symmetric, bounded 0–200; champion criterion (weekly)
WAPE	$\Sigma \text{forecast} - \text{actual} / \Sigma \text{actual} \times 100$	volume-weighted error, the usual planning KPI

Metric	Definition	Reading
Accuracy %	100 – WAPE (floored at 0)	the 'forecast accuracy' number to quote
Bias %	$(\Sigma \text{forecast} - \Sigma \text{actual}) / \Sigma \text{actual} \times 100$	negative = under-forecast

4.6 Back-test design and champion selection

Folds are aligned to calendar months so that monthly metrics use complete months: the evaluation end is the last complete month; fold k trains on all data up to (evaluation end – 3-k months) and forecasts the following BACKTEST_HORIZON_MONTHS = 3 months. With three folds on the current data the cutoffs are 31 Mar, 30 Jun and 30 Sep 2021 – the second fold is the hardest one because the June 2021 break sits inside its training window with only a few weeks of the new regime. The champion is the candidate with the lowest average **weekly sMAPE** (about 13 scored points per fold, versus 3 for monthly – the more stable criterion). It is re-selected on every run.

5. Results on the current data

Champion on the current data: **prophet_ridge_blend**. Weekly grain, averaged over the three folds (sorted by sMAPE):

Model	Accuracy %	sMAPE %	MAPE %	WAPE %	MAE	RMSE	Bias %
prophet_ridge_blend	83.2	17.5	15.5	16.8	36,984	47,350	-10.7
prophet_holt_ridge_blend	82.5	18.2	16.0	17.5	39,854	51,716	-14.2
prophet_holt_blend	82.3	18.6	16.2	17.7	38,976	50,785	-13.6
ridge_lags	81.9	18.9	16.7	18.1	45,350	55,276	-15.5
prophet	79.9	21.3	19.0	20.1	43,988	56,415	-5.9
moving_avg_4w	78.8	22.5	19.6	21.2	53,680	63,686	-19.0
naive_last_week	78.5	22.9	20.0	21.5	55,311	64,778	-18.8
holt_damped	77.0	24.5	21.2	23.0	58,287	68,289	-21.4
lightgbm_lags	70.5	33.1	27.1	29.5	72,177	84,390	-28.7

Monthly grain:

Model	Accuracy %	sMAPE %	MAPE %	WAPE %	MAE	RMSE	Bias %
prophet_ridge_blend	85.2	15.2	13.2	14.8	147,015	191,900	-10.5
prophet_holt_ridge_blend	84.1	16.3	14.1	15.9	160,476	212,913	-14.1
prophet_holt_blend	83.8	16.8	14.4	16.2	159,885	208,205	-13.5
ridge_lags	83.3	17.3	15.2	16.7	186,290	229,091	-15.3
prophet	83.4	17.3	15.1	16.6	160,926	228,291	-5.8
moving_avg_4w	80.3	20.8	18.0	19.7	221,525	267,386	-18.8
naive_last_week	80.2	21.0	18.1	19.8	228,007	271,914	-18.6
holt_damped	78.7	22.7	19.4	21.3	238,832	287,965	-21.2
lightgbm_lags	71.5	32.2	26.3	28.5	304,614	360,029	-28.5

- Blends beat their components: averaging Prophet's changepoint trend with the growth-rate regression removes each one's failure mode (Prophet over-reacts to short trends, the regression is slow to change direction).
- Every model under-forecasts on average (negative bias) because revenue kept growing through 2021; Prophet alone has the smallest bias but the largest week-to-week error.
- Tree models (LightGBM) are the weakest: ~120 training weeks are far too few for gradient boosting, and trees cannot extrapolate beyond the levels they have seen.
- Earlier exploratory back-tests (five folds, weekly data, before tax was included) also covered ARIMA, Theta, seasonal-naive, ETS variants and Fourier-trend regression; none beat the candidates kept in the notebook.

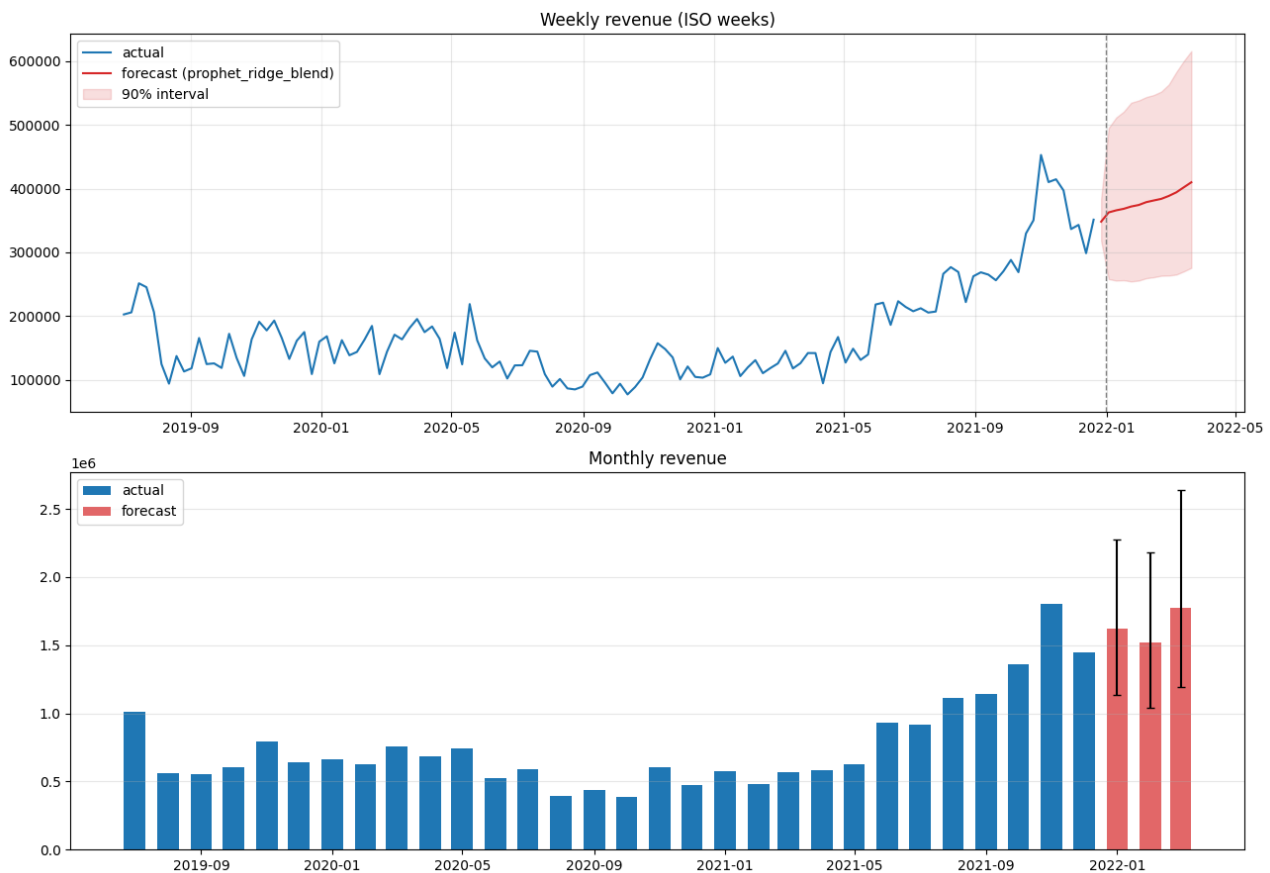
Fold-level results for the champion (prophet_ridge_blend):

Grain	Fold	Train up to	Forecast until	Accuracy %	sMAPE %	MAPE %	Bias %
monthly	1	2021-03-31	2021-06-30	75.5	25.5	20.0	-24.5
monthly	2	2021-06-30	2021-09-30	91.6	8.6	8.8	-0.5
monthly	3	2021-09-30	2021-12-31	88.5	11.4	10.8	-6.5

Grain	Fold	Train up to	Forecast until	Accuracy %	sMAPE %	MAPE %	Bias %
weekly	1	2021-03-31	2021-06-30	70.1	32.0	26.4	-24.5
weekly	2	2021-06-30	2021-09-30	90.4	9.9	9.9	-0.7
weekly	3	2021-09-30	2021-12-31	89.1	10.7	10.0	-6.8

Current forecast (revenue incl. tax):

Month	Type	Actual	Forecast	Lower 90 %	Upper 90 %
Oct 2021	actual	1,360,836			
Nov 2021	actual	1,802,234			
Dec 2021	actual	1,447,465			
Jan 2022	forecast		1,622,236	1,130,215	2,267,732
Feb 2022	forecast		1,517,198	1,037,509	2,180,747
Mar 2022	forecast		1,771,310	1,191,097	2,632,920



Weekly revenue (ISO weeks) with the 90 % interval, and monthly revenue with forecast bars and error bars – produced by section 7 of the notebook.

6. Output tables

All tables live in Gold.dbo. The first four are overwritten on every run; revenue_forecast_history_gold is appended to. Every row carries Model, HistoryEndDate, ModelRunTS, MLflowRunId and RunMode.

revenue_forecast_gold (one row per forecast day)

Column	Type	Meaning
ForecastDate	date	day
HorizonDay	int	1 = first day after the last order date
ForecastRevenue / ForecastLower / ForecastUpper	double	point forecast and interval bounds for the day
Year, Month, yyyyymm, mmmyyyy	int / string	same keys as dimdate_gold

revenue_weekly_gold (one row per ISO week, history + horizon)

Column	Type	Meaning
PeriodStart / PeriodEnd	date	Monday / Sunday of the week
IsoYear, IsoWeek, YearWeek	int / string	ISO calendar keys, e.g. 2022-W05
ActualRevenue, ActualDays	double, int	actual revenue and number of days of the week that are in the history
ForecastRevenue, ForecastLower, ForecastUpper, ForecastDays	double, int	forecast part of the week
TotalRevenue	double	ActualRevenue + ForecastRevenue – one continuous series for charts
RecordType	string	actual / forecast / mixed (the week that contains the last order date)

revenue_monthly_gold (one row per calendar month, history + horizon)

Column	Type	Meaning
PeriodStart / PeriodEnd	date	first / last day of the month
Year, Month, yyyyymm, mmmyyyy	int / string	calendar keys as in dimdate_gold
Actual..., Forecast..., TotalRevenue, RecordType		as in the weekly table

forecast_backtest_gold (one row per model × fold × grain)

Column	Type	Meaning
Model, Fold, CutoffDate, HorizonEnd, Grain, Periods		which candidate, which fold, weekly or monthly, number of scored periods
MAE, RMSE, MAPE, sMAPE, WAPE, AccuracyPct, BiasPct	double	metrics of section 4.5
IsChampion	boolean	true for the selected model

revenue_forecast_history_gold (append-only, one row per forecast month per run)

Column	Type	Meaning
PeriodStart, PeriodEnd, yyyyymm, ForecastDays		forecast month
ForecastRevenue, ForecastLower, ForecastUpper	double	what was forecast for that month at ModelRunTS
Model, HistoryEndDate, ModelRunTS, MLflowRunId, RunMode		provenance – join to revenue_monthly_gold on yyyyymm once the month is actual to measure real accuracy

7. MLflow tracking

Experiment **sales_revenue_forecast** (created automatically). Each run of the notebook produces:

- one run **backtest_<candidate>** per candidate with parameters (components, folds, horizon, Prophet settings, feature list and scaler for the ML models) and the averaged weekly/monthly metrics; fold metrics are logged as steps; tag champion = True/False;
- one run **champion_<candidate>** with the final parameters, back-test metrics, forecast_total_revenue, the CSV artefacts forecast_daily / forecast_weekly / forecast_monthly / backtest, and the fitted component models;
- registered models **sales_revenue_forecaster_prophet** (prophet flavour), ... **holt_damped** (statsmodels flavour) and ... **ridge_lags** / ... **lightgbm_lags** (sklearn pipeline incl. the scaler), depending on the champion's components. Naive components are stored as a JSON artefact.

Model logging is wrapped in try/except so that an MLflow flavour problem never fails the pipeline. Autologging is disabled to keep the runs clean.

8. How to run

8.1 Fabric portal

- Import Revenue_Forecast_Notebook.ipynb into the workspace (Workspace → Import → Notebook). The metadata pre-attaches the Gold lakehouse as default; verify it in the lakehouse explorer.
- Run all. The first cell installs Prophet (≈1 minute) and restarts the interpreter, the rest runs in well under two minutes on trial capacity.

8.2 Pipeline

- Add a third Notebook activity after Gold_Notebook connected **On success**: Silver_Notebook → Gold_Notebook → Revenue_Forecast_Notebook.
- Prophet dependency: inline %pip is disabled in pipeline runs by default. Either (recommended) create a Fabric **Environment** item, add the PyPI library prophet, publish and attach it to the notebook – the guarded first cell then does nothing – or add the Boolean base parameter **_inlineInstallationEnabled = true** to the notebook activity.
- Optional: mark the parameters cell as the notebook's parameter cell and override HORIZON_MONTHS, INCLUDE_TAX etc. from the activity.
- Schedule the pipeline (e.g. weekly). Every run retrains, re-selects the champion, refreshes the tables and appends to the history table.

8.3 VS Code – Option A: Fabric Data Engineering extension (runs on Fabric Spark)

- Install the **Fabric Data Engineering** extension, sign in, select the workspace (or press **Open in VS Code** on the notebook page in the portal).
- Choose the **Microsoft Fabric Runtime** kernel and run. The code executes on Fabric Spark, so spark exists, RUN_MODE = fabric-spark and behaviour is identical to the portal, with VS Code as editor and debugger.

8.4 VS Code – Option B: local Python kernel (reads OneLake directly)

```
python -m venv .venv && .\.venv\Scripts\activate # Linux/macOS: source .venv/bin/activate
pip install -r requirements-local.txt
az login # Azure CLI sign-in (browser prompt is the fallback)
```

- Open the notebook, pick the .venv kernel, run all. No spark → RUN_MODE = local.
- read_fact_daily() reads abfss://<workspace-id>@onelake.dfs.fabric.microsoft.com/<gold-lakehouse-id>/Tables/dbo/factsales_gold with the deltalake library and an Entra ID token from azure-identity (scope https://storage.azure.com/.default). This is a live Delta read: the latest committed version of the table, every time.
- write_table() writes the result tables back to OneLake with write_deltalake; they show up in the Gold lakehouse like any other table.
- MLflow: with synapseml-mlflow installed and LOCAL_MLFLOW_TO_FABRIC = True the runs land in the workspace experiment (tracking URI sds://api.fabric.microsoft.com/v1/workspaces/<workspace-id>/mlflow); otherwise in ./mlruns.
- Requirements: Member/Contributor role on the workspace, network access to onelake.dfs.fabric.microsoft.com. Tokens live about an hour – re-run the cell if a long session fails on auth. For unattended local runs use a service principal via AZURE_TENANT_ID / AZURE_CLIENT_ID / AZURE_CLIENT_SECRET.

	Option A (Fabric runtime kernel)	Option B (local kernel)
compute	Fabric capacity	your machine
data access	Spark, tables by name	delta-rs over OneLake, by workspace / lakehouse id
freshness	live	live
libraries	Fabric runtime + %pip / Environment	requirements-local.txt
best for	production-identical runs, pipeline debugging	fast iteration, no capacity usage

9. Power BI

- Add revenue_weekly_gold, revenue_monthly_gold, forecast_backtest_gold and revenue_forecast_history_gold to the Gold semantic model (Direct Lake).
- **History + forecast chart**: line/column visual on PeriodStart with TotalRevenue; add ForecastLower and ForecastUpper as an error band or two dotted lines; colour by RecordType.
- **KPI cards**: next-month forecast = SUM(ForecastRevenue) filtered to the first forecast month; forecast accuracy = AccuracyPct of the champion at the monthly grain from forecast_backtest_gold (IsChampion = true, Grain = monthly).

- **Model leaderboard:** matrix on forecast_backtest_gold with Model on rows, Grain on columns, averages of SMAPE / AccuracyPct.
- **Forecast accuracy over time:** relate revenue_forecast_history_gold[yyyymm] to revenue_monthly_gold[yyyymm]; measure Forecast error % = $\text{DIVIDE}(\text{SUM}(\text{history}[\text{ForecastRevenue}]) - \text{SUM}(\text{monthly}[\text{ActualRevenue}]), \text{SUM}(\text{monthly}[\text{ActualRevenue}]))$ sliced by ModelRunTS.

10. Operations, monitoring and troubleshooting

- **Retraining:** every run retrains from scratch (seconds) – there is nothing to 'update'. Re-run after new bronze files arrive.
- **Monitoring:** watch AccuracyPct of the champion in forecast_backtest_gold and, once months become actual, the realised error from the history table. A champion switch between runs is normal.
- **Reproducibility:** pin prophet in the Environment; Prophet's optimiser has a random start so point forecasts can move by a fraction of a percent between runs.

Symptom	Cause / fix
ModuleNotFoundError: prophet in a pipeline run	Inline %pip is off in pipelines. Attach an Environment with prophet or set _inlineInstallationEnabled = true on the activity.
Table or view not found: Gold.dbo.factsales_gold	Gold lakehouse not attached as default, or names changed. Fix LAKEHOUSE_NAME / SCHEMA / FACT_TABLE in the parameter cell.
A year is missing in the rows-per-year check	Date parsing in Silver_Notebook (2021 file uses M/d/yyyy). Fix the bronze file or the Silver read options.
Prophet warns about yearly seasonality with < 2 years	Informational – the training window of a fold is shorter than two years. Disappears as history grows.
Local mode: 403 / authorization error on OneLake	Account lacks workspace access, wrong ids, or token expired. az login again; check WORKSPACE_ID / GOLD_LAKEHOUSE_ID.
Local mode: MLflow 'filesystem tracking backend is in maintenance mode'	MLflow 3 with the file store – the notebook sets MLFLOW_ALLOW_FILE_STORE; otherwise install mlflow-skinny<=2.22.2 as in requirements-local.txt.
Local mode: table written but not visible in the lakehouse	Refresh the lakehouse view; tables must be under Tables/<schema>/<name> (the notebook uses Tables/dbo).
Champion has changed since the last run	Expected: the benchmark is re-run on the latest data. Compare backtest_* runs in the experiment.
Very wide intervals	Only 2.5 years of noisy weekly totals; intervals are empirical. Lower INTERVAL_WIDTH (e.g. 0.8) or wait for more history.

11. Roadmap / ideas

- **Per-category forecasts** (bikes vs. accessories/clothing, or per product family) by joining dimproduct_gold and looping the same candidates per series; sum for the total.
- **External drivers:** promotions, price changes, public holidays (Prophet add_country_holidays, or as ML features) once such data exists.
- **Hyper-parameter search** for Prophet (changepoint_prior_scale, seasonality mode) and the ML pipelines inside the notebook, logged to MLflow.
- **More candidates when history exceeds three years:** seasonal ETS / SARIMA / TBATS need at least two full seasonal cycles per fold; add folds as well.
- **Data-quality gate** in Silver_Notebook: explicit date formats per file, reject rows with null dates, keep an order-line key in factsales_gold.
- **Alerting:** a Data Activator rule on revenue_monthly_gold when an actual month closes outside its forecast band.
- **Environment item** with pinned versions (prophet, lightgbm) for fully reproducible pipeline runs.

Appendix A – parameter reference

Parameter	Default	Meaning
WORKSPACE_ID / GOLD_LAKEHOUSE_ID	workspace / Gold ids	used by local mode (OneLake path) and for MLflow-to-Fabric from a laptop
LAKEHOUSE_NAME / SCHEMA	Gold / dbo	three-part table names in Spark, path under Tables/ locally
FACT_TABLE	factsales_gold	input fact table
FORECAST_TABLE, WEEKLY_TABLE, MONTHLY_TABLE, BACKTEST_TABLE, HISTORY_TABLE	revenue_forecast_gold, revenue_weekly_gold, revenue_monthly_gold, forecast_backtest_gold, revenue_forecast_history_gold	output tables
EXPERIMENT_NAME / REGISTERED_MODEL	sales_revenue_forecast / sales_revenue_forecaster	MLflow experiment and registered-model prefix
INCLUDE_TAX	True	Revenue = Quantity × UnitPrice (+ Tax)
HORIZON_MONTHS	3	forecast the rest of the current month + this many full months
BACKTEST_FOLDS / BACKTEST_HORIZON_MONTHS	3 / 3	rolling-origin folds and their length
INTERVAL_WIDTH	0.90	prediction-interval width
KEEP_FORECAST_HISTORY	True	append each run's monthly forecast to the history table
LOCAL_MLFLOW_TO_FABRIC / LOCAL_MLFLOW_FALLBACK	True / file:./mlruns	local mode only: log to the Fabric workspace via synapseml-mlflow, else to a local folder

Appendix B – requirements-local.txt

```
# Local VS Code environment for Revenue_Forecast_Notebook.ipynb (RUN_MODE = "local")
pandas>=2.0
numpy
scipy
scikit-learn
statsmodels
prophet
lightgbm
matplotlib
ipykernel
# live access to OneLake (Delta tables) from your laptop
deltalake>=0.20
azure-identity
# MLflow: log to the Fabric workspace from outside Fabric (Microsoft's synapseml-mlflow plugin)
# if you don't need that, replace these two lines with plain `mlflow` and set LOCAL_MLFLOW_TO_FABRIC
  = False
mlflow-skinny<=2.22.2
synapseml-mlflow
```

Appendix C – full notebook source

Cells in order. Markdown cells are shown as text, code cells as code; long lines are wrapped with a 4-space continuation indent.

Cell 0 – markdown

```
# Revenue forecasting on the Gold layer (weekly + monthly)

...

Bronze (sales csv) ■Silver_Notebook■■ Silver.sales_silver ■Gold_Notebook■■ Gold.factsales_gold + dims
                                                                ■ this notebook
                                                                ▼
    Gold.revenue_forecast_gold    daily forecast (latest run)          Gold.revenue_weekly_gold
actual + forecast per ISO week
    Gold.forecast_backtest_gold    benchmark of every candidate model    Gold.revenue_monthly_gold
actual + forecast per month
    Gold.revenue_forecast_history_gold    monthly forecast of every run (append) for accuracy tracking
MLflow experiment `sales_revenue_forecast` + registered model(s)

...

**Two ways to run it - both read the *live* lakehouse tables, nothing is copied**

| | How | What happens |
|---|---|---|
| **Fabric** (portal, pipeline, or VS Code with the *Fabric Data Engineering* extension and the
  **Microsoft Fabric Runtime** kernel) | `spark` is available → `RUN_MODE = "fabric-spark"` |
  aggregation runs in Spark, tables are written with `saveAsTable`, MLflow logs to the workspace
  experiment |
| **Local VS Code** (any Python kernel, no Spark) | `spark` is missing → `RUN_MODE = "local"` | reads
  `factsales_gold` straight from OneLake with `deltalake` + `azure-identity` (Delta protocol, always
  the latest committed version), writes the result tables back to OneLake, logs MLflow to Fabric
  through `synapseml-mlflow` (falls back to `./mlruns`) |

Local setup: `pip install -r requirements-local.txt`, sign in once with `az login` (or let the browser
prompt appear), open this file in VS Code, pick your Python kernel and run all. Details in
`README_VSCode.md`.

**What it does**
1. Builds the daily revenue series (`Quantity × UnitPrice + Tax`).
2. Benchmarks 9 candidate forecasters with rolling-origin back-tests (train up to a month-end cutoff,
  forecast the next 3 months) and scores them on **weekly** and **monthly** totals: MAE, RMSE, MAPE,
  SMAPE, WAPE, Accuracy (= 100 - WAPE) and bias.
3. Picks the champion (lowest average weekly SMAPE), retrains on all history, forecasts the rest of the
  current month + `HORIZON_MONTHS` full months.
4. Writes daily / weekly / monthly tables + the benchmark table, logs runs, metrics and models to
  MLflow.

**Pipeline note** - Prophet is not in the default Fabric runtime. Interactively the first cell installs
it; for pipeline runs attach an *Environment* that contains `prophet`, or add the notebook-activity
parameter `_inlineInstallationEnabled = true` (Boolean).
```

Cell 1 – code

```
# Only Prophet is missing from the default Fabric runtime (statsmodels, scikit-learn, lightgbm, mlflow
  are included).
# KEEP THIS AS THE FIRST CELL: in Fabric, %pip restarts the Python interpreter.
# Locally, install everything with: pip install -r requirements-local.txt
try:
    import prophet # noqa: F401
    print("prophet", prophet.__version__, "available")
except ImportError:
    %pip install prophet -q
```

Cell 2 – parameters cell

```
# ----- Parameters (mark as "parameter cell" to override from a pipeline) -----
WORKSPACE_ID      = "1dca1c65-3ce2-483e-a7fb-baf7fca28e9f" # used by local mode (OneLake path) and
  MLflow-to-Fabric
GOLD_LAKEHOUSE_ID = "a89b98b2-b7b2-477d-bc9c-9005e3b396aa"
LAKEHOUSE_NAME    = "Gold"
SCHEMA            = "dbo"

FACT_TABLE        = "factsales_gold" # input (Gold_Notebook)
FORECAST_TABLE    = "revenue_forecast_gold" # output: daily forecast of the latest run
WEEKLY_TABLE      = "revenue_weekly_gold" # output: actual + forecast revenue per ISO week
MONTHLY_TABLE     = "revenue_monthly_gold" # output: actual + forecast revenue per month
BACKTEST_TABLE    = "forecast_backtest_gold" # output: benchmark metrics per model / fold / grain
HISTORY_TABLE     = "revenue_forecast_history_gold" # output: monthly forecast of EVERY run (append) ->
  track accuracy over time
KEEP_FORECAST_HISTORY = True

EXPERIMENT_NAME   = "sales_revenue_forecast"
REGISTERED_MODEL  = "sales_revenue_forecaster" # prefix of the registered ML model item(s)

INCLUDE_TAX       = True # Revenue = Quantity * UnitPrice + Tax
HORIZON_MONTHS    = 3 # forecast the rest of the current month + this many full months
```

```

BACKTEST_FOLDS      = 3          # rolling-origin folds (each = train to a month-end, forecast the next
    BACKTEST_HORIZON_MONTHS)
BACKTEST_HORIZON_MONTHS = 3
INTERVAL_WIDTH      = 0.90      # prediction-interval width

# local (VS Code python kernel) only
LOCAL_MLFLOW_TO_FABRIC = True    # log to the Fabric workspace via synapseml-mlflow; needs `pip
    install synapseml-mlflow "mlflow-skinny<=2.22.2"`
LOCAL_MLFLOW_FALLBACK = "file:./mlruns" # used when the plugin is not installed

```

Cell 3 – markdown

```
## 1. Connect to the Gold lakehouse (Fabric Spark or OneLake from your laptop)
```

Cell 4 – code

```

import os, warnings
import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

try:
    spark          # exists in Fabric Spark sessions (portal, pipeline, VS Code "Microsoft
        Fabric Runtime" kernel)
    RUN_MODE = "fabric-spark"
except NameError:
    RUN_MODE = "local"
print("RUN_MODE =", RUN_MODE)

GOLD_TABLE_ROOT =
    f"abfss://{WORKSPACE_ID}@onelake.dfs.fabric.microsoft.com/{GOLD_LAKEHOUSE_ID}/Tables/{SCHEMA}"

def spark_table_name(table):
    return ".".join(p for p in (LAKEHOUSE_NAME, SCHEMA, table) if p)

def onelake_path(table):
    return f"{GOLD_TABLE_ROOT}/{table}"

if RUN_MODE == "local":
    from deltalake import DeltaTable, write_deltalake

    def onelake_storage_options():
        if not GOLD_TABLE_ROOT.startswith("abfss:///"):
            return {}
        from azure.identity import DefaultAzureCredential # az login / VS Code sign-in / browser
        prompt
        cred = DefaultAzureCredential(exclude_interactive_browser_credential=False)
        token = cred.get_token("https://storage.azure.com/.default").token
        return {"bearer_token": token, "use_fabric_endpoint": "true"}

    def read_fact_daily():
        # Always queries the live Delta table (latest committed version) - nothing is cached or copied.
        if RUN_MODE == "fabric-spark":
            from pyspark.sql import functions as F
            fact = spark.read.table(spark_table_name(FACT_TABLE))
            revenue = F.col("Quantity") * F.col("UnitPrice") + (F.col("Tax") if INCLUDE_TAX else F.lit(0.0))
            pdf = (fact.filter(F.col("OrderDate").isNotNull())
                .groupBy("OrderDate")
                .agg(F.sum(revenue).alias("Revenue"), F.count("*").alias("OrderLines"))
                .orderBy("OrderDate")
                .toPandas())
        else:
            dt = DeltaTable(onelake_path(FACT_TABLE), storage_options=onelake_storage_options())
            pdf = dt.to_pandas(columns=["OrderDate", "Quantity", "UnitPrice",
                "Tax"]).dropna(subset=["OrderDate"])
            pdf["Revenue"] = (pdf["Quantity"].astype(float) * pdf["UnitPrice"].astype(float)
                + (pdf["Tax"].astype(float) if INCLUDE_TAX else 0.0))
            pdf = pdf.groupby("OrderDate").agg(Revenue=("Revenue", "sum"), OrderLines=("Revenue",
                "size")).reset_index()
            pdf["OrderDate"] = pd.to_datetime(pdf["OrderDate"])
            return pdf.sort_values("OrderDate").reset_index(drop=True)

    def write_table(pdf, table, date_cols=(), mode="overwrite"):
        # Writes a pandas DataFrame to a Delta table in the Gold lakehouse (mode = "overwrite" or "append").
        pdf = pdf.copy()
        if RUN_MODE == "fabric-spark":
            from pyspark.sql import functions as F
            sdf = spark.createDataFrame(pdf)
            for c in date_cols:
                sdf = sdf.withColumn(c, F.to_date(c))
            writer = sdf.write.format("delta").mode(mode)
            writer = writer.option("overwriteSchema", "true") if mode == "overwrite" else
                writer.option("mergeSchema", "true")

```

```

        writer.saveAsTable(spark_table_name(table))
    else:
        for c in date_cols:
            pdf[c] = pd.to_datetime(pdf[c]).dt.date
        for c in pdf.columns:
            # Delta stores microsecond timestamps
            if pd.api.types.is_datetime64_any_dtype(pdf[c]):
                pdf[c] = pdf[c].astype("datetime64[us, UTC]" if getattr(pdf[c].dt, "tz", None) is not
                None else "datetime64[us]")
            write_deltalake(onelake_path(table), pdf, mode=mode, schema_mode="overwrite" if mode ==
            "overwrite" else "merge",
                            storage_options=onelake_storage_options())
        print(f"'{mode}' if mode == 'append' else 'wrote' {len(pdf):,} rows -> {table}")

```

Cell 5 – code

```

# ---- MLflow tracking: Fabric experiment in both modes when possible ----
import mlflow

```

```

if RUN_MODE == "local":
    tracking = None
    if LOCAL_MLFLOW_TO_FABRIC:
        try:
            from fabric.analytics.environment.credentials import
            SetFabricAnalyticsDefaultTokenCredentialsGlobally
            from azure.identity import DefaultAzureCredential
            SetFabricAnalyticsDefaultTokenCredentialsGlobally(
                credential=DefaultAzureCredential(exclude_interactive_browser_credential=False))
            tracking = f"sds://api.fabric.microsoft.com/v1/workspaces/{WORKSPACE_ID}/mlflow"
        except ImportError:
            print("synapseml-mlflow not installed -> logging to", LOCAL_MLFLOW_FALLBACK)
    if tracking is None:
        os.environ.setdefault("MLFLOW_ALLOW_FILE_STORE", "true")
        tracking = LOCAL_MLFLOW_FALLBACK
    mlflow.set_tracking_uri(tracking)

mlflow.set_experiment(EXPERIMENT_NAME)
mlflow.autolog(disable=True) # everything is logged explicitly
print("MLflow tracking URI:", mlflow.get_tracking_uri())

```

Cell 6 – markdown

```

## 2. Daily, weekly and monthly revenue history

```

Cell 7 – code

```

daily_pdf = read_fact_daily()
daily = (daily_pdf.set_index("OrderDate")["Revenue"].astype(float)
        .sort_index().asfreq("D", fill_value=0.0)) # continuous calendar, 0 on days
        without orders
LAST_DATE = daily.index.max()
is_month_end = LAST_DATE == LAST_DATE + pd.offsets.MonthEnd(0)
FORECAST_END = LAST_DATE + pd.offsets.MonthEnd(HORIZON_MONTHS if is_month_end else HORIZON_MONTHS + 1)
HORIZON_DAYS = int((FORECAST_END - LAST_DATE).days)

print(f"{len(daily)} days | {daily.index.min().date()} -> {LAST_DATE.date()} | total revenue
      {daily.sum():.0f}")
print(f"forecast window: {(LAST_DATE + pd.Timedelta(days=1)).date()} -> {FORECAST_END.date()}
      ({HORIZON_DAYS} days)\n")
print("Rows per year (if a year is missing, check date parsing in Silver Notebook):")
print(daily_pdf.assign(Year=daily_pdf["OrderDate"].dt.year).groupby("Year")
      .agg(days_with_sales=("OrderDate", "nunique"), order_lines=("OrderLines", "sum"),
      revenue=("Revenue", "sum")).round(0))

def to_periods(frame, grain):
    # frame: DataFrame indexed by day. Returns per-period sums + number of days covered, indexed by
    # period start.
    rule = "W-SUN" if grain == "weekly" else "MS" # ISO weeks (Mon-Sun) / calendar months
    out = frame.resample(rule).sum()
    out["Days"] = frame.iloc[:, 0].resample(rule).count()
    if grain == "weekly":
        out.index = out.index - pd.Timedelta(days=6) # label by Monday
        out.index.name = "PeriodStart"
    return out

actual_weekly = to_periods(daily.to_frame("ActualRevenue"), "weekly").rename(columns={"Days":
    "ActualDays"})
actual_monthly = to_periods(daily.to_frame("ActualRevenue"), "monthly").rename(columns={"Days":
    "ActualDays"})
print("\nLast 6 weeks (Mon-Sun):"); print(actual_weekly.tail(6).round(0))
print("\nLast 6 months:"); print(actual_monthly.tail(6).round(0))

```

Cell 8 – markdown

```

## 3. Candidate models (with scaling where it matters)

```

```

| candidate | type | scaling |

```

```

|---|---|---|
| `naive_last_week`, `moving_avg_4w` | baselines | - |
| `holt_damped` | Holt's damped-trend exponential smoothing on weekly totals | scale-invariant |
| `prophet` | piecewise-linear trend with changepoints + yearly seasonality, daily data | scale-
invariant (log-target was tested and hurt) |
| `ridge_lags` | Ridge regression on lagged weekly growth rates | **StandardScaler** inside the pipeline
(required for a penalised linear model) + **log target** |
| `lightgbm_lags` | gradient-boosted trees on the same features | StandardScaler in the pipeline for
consistency (trees are insensitive to it) + log target |
| `prophet_holt_blend`, `prophet_ridge_blend`, `prophet_holt_ridge_blend` | averages of components | - |

```

The ML models predict weekly **log-growth** ($\log(1+y_t) - \log(1+y_{t-1})$) rather than the raw level: it makes the target stationary, lets the models extrapolate the 2021 growth, and removes the scale of revenue from the problem. Intervals for the baselines and ML models use the empirical spread of h-week-ahead changes in log revenue measured on the training window.

Cell 9 – code

```

import logging
from scipy.stats import norm
from prophet import Prophet
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from lightgbm import LGBMRegressor

logging.getLogger("prophet").setLevel(logging.ERROR)
logging.getLogger("cmdstanpy").disabled = True
Z = float(norm.ppf(1 - (1 - INTERVAL_WIDTH) / 2))

PROPHET_PARAMS = dict(growth="linear", yearly_seasonality=True, weekly_seasonality=False,
                      daily_seasonality=False,
                      changepoint_prior_scale=0.5, changepoint_range=0.9, interval_width=INTERVAL_WIDTH)
# Lagged growth only: level-vs-trend and calendar features were tested and hurt (they are learned from a
# single prior year).
# growth_mean4 is left out on purpose: it is exactly the mean of growth_lag1..4 (perfect collinearity,
# identical forecasts without it).
# Remaining features are only mildly correlated (adjacent lags about -0.4, VIF <= 1.7).
ML_FEATURE_NAMES = ["growth_lag1", "growth_lag2", "growth_lag3", "growth_lag4", "growth_lag8"]

def weekly_bins(series):
    # complete 7-day bins ending on the last date of the series (used for modelling, not for reporting)
    anchor = "W-" + ["MON", "TUE", "WED", "THU", "FRI", "SAT", "SUN"][series.index[-1].dayofweek]
    w = series.resample(anchor).sum()
    if (w.index[0] - pd.Timedelta(days=6)) < series.index[0]:
        w = w.iloc[1:]
    return w

def weekly_to_daily(point, lower, upper, start, horizon):
    idx = pd.date_range(start, periods=horizon, freq="D")
    spread = lambda x: np.repeat(np.asarray(x, float) / 7.0, 7)[:horizon]
    return pd.DataFrame({"yhat": spread(point), "yhat_lower": spread(lower), "yhat_upper":
                        spread(upper)}, index=idx).clip(lower=0.0)

def horizon_log_std(weekly, max_h=52, min_obs=10):
    # empirical std of the h-week change in log revenue, h = 1..max_h (a random walk would grow like
    # sqrt(h);
    # weekly totals are noisy but that noise is transient, so the measured growth is much slower)
    l = np.log1p(weekly); out = []
    for h in range(1, max_h + 1):
        d = (l.shift(-h) - l).dropna()
        out.append(float(d.std()) if len(d) >= min_obs else (out[-1] * np.sqrt(h / len(out))) if out else
                    0.3))
    return np.maximum.accumulate(np.array(out)) # non-decreasing in h

def growth_band(point, hstd, horizon_weeks):
    s = np.resize(hstd, horizon_weeks) if horizon_weeks > len(hstd) else hstd[:horizon_weeks]
    return point * np.exp(-Z * s), point * np.exp(Z * s)

# ---- baselines -----
def make_naive(n_weeks):
    def fit(series):
        w = weekly_bins(series)
        return {"level": float(w.iloc[-n_weeks:].mean()), "hstd": horizon_log_std(w)}
    def predict(model, start, horizon):
        n = int(np.ceil(horizon / 7)); point = np.repeat(model["level"], n)
        lo, hi = growth_band(point, model["hstd"], n)
        return weekly_to_daily(point, lo, hi, start, horizon)
    return fit, predict

# ---- Holt damped trend (weekly) -----
def fit_holt(series):

```

```

    return ExponentialSmoothing(weekly_bins(series).values, trend="add",
                                damped_trend=True).fit(optimized=True)

def predict_holt(model, start, horizon):
    n = int(np.ceil(horizon / 7))
    point = np.clip(model.forecast(n), 0, None)
    sims = model.simulate(n, repetitions=500, error="add", random_state=42)
    lo = np.clip(np.quantile(sims, (1 - INTERVAL_WIDTH) / 2, axis=1), 0, None)
    hi = np.quantile(sims, 1 - (1 - INTERVAL_WIDTH) / 2, axis=1)
    return weekly_to_daily(point, lo, hi, start, horizon)

# ---- Prophet (daily) -----
def fit_prophet(series):
    m = Prophet(**PROPHET_PARAMS)
    m.fit(pd.DataFrame({"ds": series.index, "y": series.values}))
    return m

def predict_prophet(model, start, horizon):
    future = pd.DataFrame({"ds": pd.date_range(start, periods=horizon, freq="D")})
    return model.predict(future).set_index("ds")[["yhat", "yhat_lower", "yhat_upper"]].clip(lower=0.0)

# ---- feature-based ML models (weekly log-growth target, StandardScaler + estimator pipeline)
-----
def ml_features(logy):
    g = logy.diff()
    X = pd.DataFrame(index=logy.index)
    for lag in (1, 2, 3, 4, 8):
        X[f"growth_lag{lag}"] = g.shift(lag)
    X["growth_mean4"] = g.shift(1).rolling(4).mean()
    X["growth_mean13"] = g.shift(1).rolling(13).mean()
    X["level_vs_13w"] = logy.shift(1) - logy.shift(1).rolling(13).mean() # available for
    experiments
    woy = X.index.isocalendar().week.astype(float).values
    X["woy_sin"], X["woy_cos"] = np.sin(2 * np.pi * woy / 52.18), np.cos(2 * np.pi * woy / 52.18)
    X["month"] = X.index.month
    return X[ML_FEATURE_NAMES], g

def make_ml(estimator_factory):
    def fit(series):
        logy = np.log1p(weekly_bins(series))
        X, g = ml_features(logy)
        mask = X.notna().all(axis=1) & g.notna()
        pipe = Pipeline([("scale", StandardScaler()), ("model", estimator_factory())])
        pipe.fit(X[mask], g[mask])
        return {"pipe": pipe, "logy": logy, "hstd": horizon_log_std(np.expm1(logy))}
    def predict(model, start, horizon):
        logy, pipe = model["logy"].copy(), model["pipe"]
        n = int(np.ceil(horizon / 7)); preds = []
        for _ in range(n): # recursive multi-step
            nxt = logy.index[-1] + pd.Timedelta(weeks=1)
            logy.loc[nxt] = np.nan
            X, _ = ml_features(logy)
            logy.loc[nxt] = logy.iloc[-2] + float(pipe.predict(X.iloc[[-1]])[0])
            preds.append(logy.iloc[-1])
        point = np.expm1(np.array(preds))
        lo, hi = growth_band(point, model["hstd"], n)
        return weekly_to_daily(point, lo, hi, start, horizon)
    return fit, predict

COMPONENTS = {
    "naive_last_week": make_naive(1),
    "moving_avg_4w": make_naive(4),
    "holt_damped": (fit_holt, predict_holt),
    "prophet": (fit_prophet, predict_prophet),
    "ridge_lags": make_ml(lambda: Ridge(alpha=1.0)),
    "lightgbm_lags": make_ml(lambda: LGBMRegressor(n_estimators=100, learning_rate=0.02, num_leaves=3,
                                                    min_child_samples=10, verbose=-1,
                                                    random_state=42)),
}

CANDIDATES = {**{name: [name] for name in COMPONENTS},
               "prophet_holt_blend": ["prophet", "holt_damped"],
               "prophet_ridge_blend": ["prophet", "ridge_lags"],
               "prophet_holt_ridge_blend": ["prophet", "holt_damped", "ridge_lags"]}

def blend(component_preds, names):
    return sum(component_preds[n] for n in names) / len(names)

def score(actual, forecast):
    a, f = np.asarray(actual, float), np.asarray(forecast, float)
    err = f - a
    wape = 100 * np.sum(np.abs(err)) / max(np.sum(np.abs(a)), 1e-9)

```

```

return {"MAE": float(np.mean(np.abs(err))),
        "RMSE": float(np.sqrt(np.mean(err ** 2))),
        "MAPE": float(100 * np.mean(np.abs(err) / np.maximum(np.abs(a), 1e-9))),
        "sMAPE": float(100 * np.mean(2 * np.abs(err) / np.maximum(np.abs(a) + np.abs(f), 1e-9))),
        "WAPE": float(wape),
        "AccuracyPct": float(max(0.0, 100 - wape)),
        "BiasPct": float(100 * np.sum(err) / max(np.sum(a), 1e-9))}

```

Cell 10 – markdown

4. Benchmark: rolling-origin back-test on weekly and monthly totals
Each fold trains on everything up to a month-end cutoff and forecasts the next 3 calendar months; the folds are the last three quarters of history. The champion is the lowest average **weekly sMAPE** (weekly has ~13 points per fold, monthly only 3, so it is the more stable criterion).

Cell 11 – code

```

import time

eval_end = LAST_DATE if is_month_end else LAST_DATE + pd.offsets.MonthEnd(-1)
cutoffs = [eval_end + pd.offsets.MonthEnd(-BACKTEST_HORIZON_MONTHS * k) for k in range(BACKTEST_FOLDS,
0, -1)]

rows = []
for fold, cutoff in enumerate(cutoffs, start=1):
    horizon_end = cutoff + pd.offsets.MonthEnd(BACKTEST_HORIZON_MONTHS)
    train = daily[:cutoff]
    actual = daily[cutoff + pd.Timedelta(days=1): horizon_end]
    start = cutoff + pd.Timedelta(days=1)

    preds = {}
    for comp, (fit_fn, predict_fn) in COMPONENTS.items():
        t0 = time.time()
        preds[comp] = predict_fn(fit_fn(train), start, len(actual))
        print(f"fold {fold} | train to {cutoff.date()} | {comp:<16} {time.time() - t0:5.1f}s")

    actual_w, actual_m = weekly_bins(actual), actual.resample("MS").sum()
    for cand, comps in CANDIDATES.items():
        fc = blend(preds, comps)["yhat"]
        for grain, a, f in (("weekly", actual_w, weekly_bins(fc)), ("monthly", actual_m,
fc.resample("MS").sum())):
            rows.append({"Model": cand, "Fold": fold, "CutoffDate": cutoff.date(), "HorizonEnd":
horizon_end.date(),
                        "Grain": grain, "Periods": len(a), **score(a.values, f.values)})

backtest = pd.DataFrame(rows)
METRICS = ["MAE", "RMSE", "MAPE", "sMAPE", "WAPE", "AccuracyPct", "BiasPct"]
leaderboard = (backtest.groupby(["Grain", "Model"])[METRICS].mean()
                .unstack("Grain").swaplevel(axis=1).sort_index(axis=1))
leaderboard = leaderboard.sort_values(("weekly", "sMAPE"))
CHAMPION = leaderboard.index[0]
backtest["IsChampion"] = backtest["Model"].eq(CHAMPION)

pd.set_option("display.width", 220)
print("\n=== Benchmark: average over folds (weekly grain) ===")
print(leaderboard["weekly"].round(1).to_string())
print("\n=== Benchmark: average over folds (monthly grain) ===")
print(leaderboard["monthly"].loc[leaderboard.index].round(1).to_string())
print(f"\nChampion: {CHAMPION}")

# one MLflow run per candidate so the benchmark is visible in the experiment
for cand, comps in CANDIDATES.items():
    with mlflow.start_run(run_name=f"backtest_{cand}"):
        mlflow.log_params({"model": cand, "components": "+".join(comps), "backtest_folds":
BACKTEST_FOLDS,
                           "backtest_horizon_months": BACKTEST_HORIZON_MONTHS, "include_tax":
INCLUDE_TAX,
                           **({f"prophet_{k}": v for k, v in PROPHET_PARAMS.items()} if "prophet" in
comps else {}),
                           **({"features": ", ".join(ML_FEATURE_NAMES), "scaler": "StandardScaler",
"target": "weekly log-growth"
                               if any(c.endswith("_lags") for c in comps) else {}))})
        for grain in ("weekly", "monthly"):
            mlflow.log_metrics({f"{grain}_{m}": float(v) for m, v in leaderboard.loc[cand,
grain].items()})
            for _, r in backtest[(backtest["Model"] == cand) & (backtest["Grain"] == grain)].iterrows():
                mlflow.log_metrics({f"fold_{grain}_sMAPE": r["sMAPE"], f"fold_{grain}_AccuracyPct":
r["AccuracyPct"]}, step=int(r["Fold"]))
            mlflow.set_tag("champion", str(cand == CHAMPION))

```

Cell 12 – markdown

5. Retrain the champion on all history → daily, weekly and monthly forecast

Cell 13 – code

```

from datetime import datetime, timezone

```

```

RUN_TS = datetime.now(timezone.utc)
FORECAST_START = LAST_DATE + pd.Timedelta(days=1)

def log_component_model(comp, model):
    try:
        name = f"{REGISTERED_MODEL}_{comp}"
        if comp == "prophet":
            mlflow.prophet.log_model(model, artifact_path=comp, registered_model_name=name)
        elif comp == "holt_damped":
            mlflow.statsmodels.log_model(model, artifact_path=comp, registered_model_name=name)
        elif comp.endswith("_lags"):
            mlflow.sklearn.log_model(model["pipe"], artifact_path=comp, registered_model_name=name)
        else:
            mlflow.log_dict({"level": model["level"]}, f"{comp}.json")
    except Exception as e:
        # a flavour problem must not fail the pipeline
        print(f"WARNING: could not log {comp} to MLflow: {e}")

with mlflow.start_run(run_name=f"champion_{CHAMPION}") as run:
    fitted, preds = {}, {}
    for comp in CANDIDATES[CHAMPION]:
        fit_fn, predict_fn = COMPONENTS[comp]
        fitted[comp] = fit_fn(daily)
        preds[comp] = predict_fn(fitted[comp], FORECAST_START, HORIZON_DAYS)
    fc = blend(preds, CANDIDATES[CHAMPION]).rename(columns={"yhat": "ForecastRevenue", "yhat_lower":
"ForecastLower", "yhat_upper": "ForecastUpper"})

    # daily table
    forecast_daily = fc.reset_index().rename(columns={"index": "ForecastDate", "ds": "ForecastDate"})
    forecast_daily.insert(1, "HorizonDay", np.arange(1, HORIZON_DAYS + 1))
    forecast_daily["Year"], forecast_daily["Month"] = forecast_daily["ForecastDate"].dt.year,
forecast_daily["ForecastDate"].dt.month
    forecast_daily["yyyymm"] = forecast_daily["ForecastDate"].dt.strftime("%Y%m")
    forecast_daily["mmmyyyy"] = forecast_daily["ForecastDate"].dt.strftime("%b-%Y")

    # weekly / monthly tables = actual history + forecast, one row per period (a period can hold both if
    it straddles LAST_DATE)
    def combined(grain):
        f = to_periods(fc, grain).rename(columns={"Days": "ForecastDays"})
        out = actual_weekly.join(f, how="outer") if grain == "weekly" else actual_monthly.join(f,
how="outer")
        out = out.fillna({"ActualRevenue": 0.0, "ActualDays": 0, "ForecastRevenue": 0.0,
"ForecastLower": 0.0, "ForecastUpper": 0.0, "ForecastDays": 0})
        out["ActualDays"], out["ForecastDays"] = out["ActualDays"].astype(int),
out["ForecastDays"].astype(int)
        out = out.reset_index()
        if grain == "weekly":
            out["PeriodEnd"] = out["PeriodStart"] + pd.Timedelta(days=6)
            iso = out["PeriodStart"].dt.isocalendar()
            out["IsoYear"], out["IsoWeek"] = iso["year"].astype(int), iso["week"].astype(int)
            out["YearWeek"] = out["IsoYear"].astype(str) + "-W" +
out["IsoWeek"].astype(str).str.zfill(2)
        else:
            out["PeriodEnd"] = out["PeriodStart"] + pd.offsets.MonthEnd(0)
            out["Year"], out["Month"] = out["PeriodStart"].dt.year, out["PeriodStart"].dt.month
            out["yyyymm"], out["mmmyyyy"] = out["PeriodStart"].dt.strftime("%Y%m"),
out["PeriodStart"].dt.strftime("%b-%Y")
        out["TotalRevenue"] = out["ActualRevenue"] + out["ForecastRevenue"] # continuous series
        for charts
            out["RecordType"] = np.select([out["ForecastDays"].eq(0), out["ActualDays"].eq(0)], ["actual",
"forecast"], "mixed")
        return out

    forecast_weekly, forecast_monthly = combined("weekly"), combined("monthly")

    for df_in (forecast_daily, forecast_weekly, forecast_monthly):
        df_["Model"], df_["HistoryEndDate"], df_["ModelRunTS"], df_["MLflowRunId"], df_["RunMode"] =
CHAMPION, LAST_DATE, RUN_TS, run.info.run_id, RUN_MODE

        mlflow.log_params({"model": CHAMPION, "components": "+".join(CANDIDATES[CHAMPION]), "include_tax":
INCLUDE_TAX,
            "history_start": str(daily.index.min().date()), "history_end":
str(LAST_DATE.date()),
            "forecast_end": str(FORECAST_END.date()), "horizon_days": HORIZON_DAYS,
"interval_width": INTERVAL_WIDTH, "run_mode": RUN_MODE})
        for grain in ("weekly", "monthly"):
            mlflow.log_metrics({f"backtest_{grain}_{m}": float(v) for m, v in leaderboard.loc[CHAMPION,
grain].items()})
            mlflow.log_metric("forecast_total_revenue", float(fc["ForecastRevenue"].sum()))
            for name, df_in in (("forecast_daily", forecast_daily), ("forecast_weekly", forecast_weekly),
("forecast_monthly", forecast_monthly), ("backtest", backtest)):
                mlflow.log_text(df_.to_csv(index=False), f"{name}.csv")
            for comp, model in fitted.items():
                log_component_model(comp, model)

print(f"champion = {CHAMPION} | forecast {FORECAST_START.date()} -> {FORECAST_END.date()} | expected

```

```

revenue {fc['ForecastRevenue'].sum():, .0f}\n")
cols = ["PeriodStart", "PeriodEnd", "RecordType", "ActualRevenue", "ForecastRevenue", "ForecastLower",
"ForecastUpper", "TotalRevenue"]
print("Weekly (last 4 actual weeks + forecast):"); print(forecast_weekly[forecast_weekly["PeriodEnd"]
>= LAST_DATE - pd.Timedelta(weeks=4)][cols].round(0).to_string(index=False))
print("\nMonthly (last 3 actual months + forecast):");
print(forecast_monthly[forecast_monthly["PeriodEnd"] >= LAST_DATE -
pd.DateOffset(months=3)][cols].round(0).to_string(index=False))

```

Cell 14 – markdown

```

## 6. Write everything to the Gold lakehouse
The four main tables are overwritten on each run (stamped with `ModelRunTS`);
`revenue_forecast_history_gold` is appended to, so every run's monthly forecast is kept and can
later be compared with the actual months (forecast accuracy over time).

```

Cell 15 – code

```

write_table(forecast_daily, FORECAST_TABLE, date_cols=("ForecastDate", "HistoryEndDate"))
write_table(forecast_weekly, WEEKLY_TABLE, date_cols=("PeriodStart", "PeriodEnd", "HistoryEndDate"))
write_table(forecast_monthly, MONTHLY_TABLE, date_cols=("PeriodStart", "PeriodEnd", "HistoryEndDate"))
write_table(backtest.assign(ModelRunTS=RUN_TS, RunMode=RUN_MODE), BACKTEST_TABLE,
date_cols=("CutoffDate", "HorizonEnd"))

if KEEP_FORECAST_HISTORY: # one row per forecast month per run -> join to actual months later to
measure real-life accuracy
hist_cols = ["PeriodStart", "PeriodEnd", "yyyymm", "ForecastDays", "ForecastRevenue",
"ForecastLower", "ForecastUpper",
"Model", "HistoryEndDate", "ModelRunTS", "MLflowRunId", "RunMode"]
write_table(forecast_monthly.loc[forecast_monthly["ForecastDays"] > 0, hist_cols], HISTORY_TABLE,
date_cols=("PeriodStart", "PeriodEnd", "HistoryEndDate"), mode="append")

```

Cell 16 – markdown

```

## 7. Charts

```

Cell 17 – code

```

import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(13, 9))

w = forecast_weekly.set_index("PeriodStart")
w = w[(w["ActualDays"] + w["ForecastDays"]) == 7] # complete weeks only (partial edge weeks
would show as dips)
hist = w[w["RecordType"] == "actual"]; fut = w[w["ForecastDays"] > 0] # a mixed week is plotted as
actual + forecast days
ax1.plot(hist.index, hist["ActualRevenue"], color="#1f77b4", label="actual")
ax1.plot(fut.index, fut["TotalRevenue"], color="#d62728", label=f"forecast ({CHAMPION})")
ax1.fill_between(fut.index, fut["ActualRevenue"] + fut["ForecastLower"], fut["ActualRevenue"] +
fut["ForecastUpper"], color="#d62728", alpha=0.15, label=f"{int(INTERVAL_WIDTH*100)}% interval")
ax1.axvline(LAST_DATE, color="grey", ls="--", lw=1); ax1.set_title("Weekly revenue (ISO weeks)");
ax1.legend(loc="upper left"); ax1.grid(alpha=0.3)

m = forecast_monthly.set_index("PeriodStart")
ax2.bar(m.index, m["ActualRevenue"], width=20, color="#1f77b4", label="actual")
ax2.bar(m.index, m["ForecastRevenue"], width=20, bottom=m["ActualRevenue"], color="#d62728", alpha=0.7,
label="forecast")
fut_m = m[m["ForecastDays"] > 0]
ax2.errorbar(fut_m.index, fut_m["ActualRevenue"] + fut_m["ForecastRevenue"],
yerr=[fut_m["ForecastRevenue"] - fut_m["ForecastLower"], fut_m["ForecastUpper"] -
fut_m["ForecastRevenue"]], fmt="none", ecolor="black", capsize=3)
ax2.set_title("Monthly revenue"); ax2.legend(loc="upper left"); ax2.grid(alpha=0.3, axis="y")
plt.tight_layout(); plt.show()

```

Cell 18 – markdown

```

## 8. Next steps
* **Pipeline** - add this notebook as a third activity after `Gold_Notebook` (On success). Handle the
Prophet dependency (Environment or `_inlineInstallationEnabled`).
* **Power BI** - `revenue_weekly_gold` / `revenue_monthly_gold` already contain actual + forecast per
period (`TotalRevenue` for a continuous line, `ForecastLower/Upper` for the band);
`forecast_backtest_gold` gives the model leaderboard (filter `Grain`); join
`revenue_forecast_history_gold` to `revenue_monthly_gold` on `yyyymm` to report how accurate past
forecasts turned out.
* **VS Code** - see `README_VSCode.md`: Fabric extension + *Microsoft Fabric Runtime* kernel to run on
Fabric Spark, or a plain Python kernel that reads OneLake directly (`RUN_MODE = local`).
* **Extensions** - per-category forecasts (bikes vs. accessories), holidays/promotions as Prophet
regressors or ML features, hyper-parameter search on the ML pipelines, more folds as history grows.

```